

```
"""
```

```
verify_executor.py – GRUS Audit Pipeline Stage 5
```

```
Specification: GRUS-STD-001-v1.0 Section 3 Stage 5
```

```
Executes the submission's verify.py in a sandboxed subprocess and captures  
stdout for residual comparison.
```

```
Published by: Green Recursive Utility Service LLC
```

```
"""
```

```
import subprocess, os, sys
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class VerifyExecutionResult:
```

```
    passed: bool
```

```
    stdout: str = ""
```

```
    stderr: str = ""
```

```
    returncode: int = -1
```

```
    runtime_seconds: float = 0.0
```

```
    failure_reason: str = ""
```

```
def execute_verify(repo_path: str, timeout_sec: int = 300) -> VerifyExecutionResult:
```

```
    verify_path = os.path.join(repo_path, "verify.py")
```

```
    if not os.path.exists(verify_path):
```

```
        return VerifyExecutionResult(passed=False,
```

```
            failure_reason="verify.py not found in repository root")
```

```
    import time
```

```
    start = time.time()
```

```
    try:
```

```
        proc = subprocess.run([sys.executable, verify_path],
```

```
                               cwd=repo_path,
```

```
                               capture_output=True, text=True,
```

```
                               timeout=timeout_sec)
```

```
    except subprocess.TimeoutExpired:
```

```
        return VerifyExecutionResult(passed=False,
```

```
            failure_reason=f"verify.py exceeded {timeout_sec}s timeout")
```

```
    except Exception as e:
```

```
        return VerifyExecutionResult(passed=False,
```

```
            failure_reason=f"verify.py execution error: {e}")
```

```
    runtime = time.time() - start
```

```
    if proc.returncode != 0:
```

```
        return VerifyExecutionResult(passed=False, stdout=proc.stdout,
```

```
            stderr=proc.stderr, returncode=proc.returncode,
```

```
            runtime_seconds=runtime,
```

```
            failure_reason=f"verify.py exited with code {proc.returncode}")
```

```
    return VerifyExecutionResult(passed=True, stdout=proc.stdout,
```

```
        stderr=proc.stderr, returncode=0, runtime_seconds=runtime)
```

```
if __name__ == "__main__":
```

```
    if len(sys.argv) < 2:
```

```
    print("Usage: python verify_executor.py <repo_path>"); sys.exit(0)
r = execute_verify(sys.argv[1])
if r.passed:
    print(f"[PASS] verify.py executed in {r.runtime_seconds:.2f}s")
    print("--- STDOUT ---"); print(r.stdout)
else:
    print(f"[FAIL] {r.failure_reason}"); sys.exit(1)
```